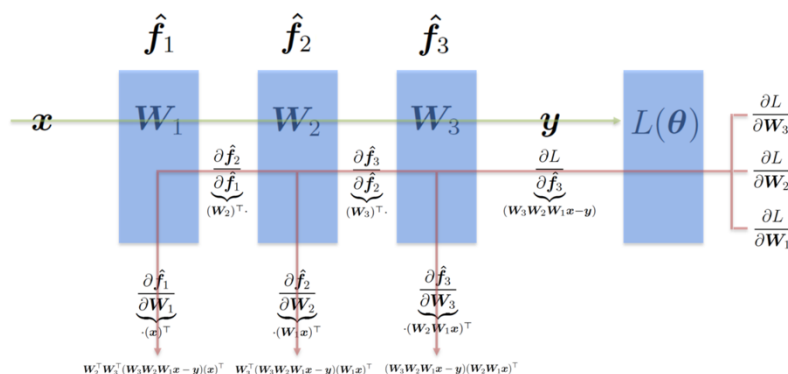
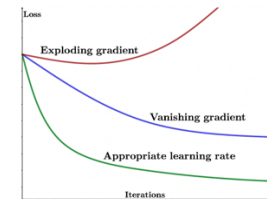


Neural Networks:

Feed Forward Neural Networks:

- The classification of a Perceptron depends on the sign of the distance $\hat{y} = \text{sign}(w^T x)$
- One perceptron is the unit of the sum of the input multiplied by the weights with the activation function
- XOR problem can't be solved with a single perceptron / single linear decision boundary
- Multi-layer perceptrons enable non-linear decision boundaries
- Universal Approximation Theorem: A network with a single hidden layer can approximate any continuous function $F(x) = \sum_{i=1}^N v_i \varphi(w_i^T x + b_i)$
- The more neurons, the better the approximation
- Improve approximation by adding non-linearity
- One-hot encoding: only one element is not 0
- Softmax activation function (=normalized exponential function) allows to treat the input as normalized probabilities (numbers between 0 and 1)
- Loss functions tell us how good the prediction of the network is, difference between actual and expected output
- Softmax loss: softmax activation + cross entropy loss for multi class problems, result in one-hot encoding vectors
- Optimization: find optimal weights for all layers -> minimize the loss function
- Use Gradient descent: follow the negative gradient with learning rate η : simple and always finds a local minimum
- Derivatives can be calculated with finite differences or with an analytic derivative
- Finite differences: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+\frac{1}{2}h) - f(x-\frac{1}{2}h)}{h}$ use $h = 1 \cdot 10^{-5}$
- Used to check analytic gradient implementation
- Analytic gradient: Ableitungsregeln wie in der Schule, computationally efficient
- Forward pass:
 - o compute activations (input * weights, sum them up and process through activation function) y from x, w
 - o compute the loss function (total error) L from y
- backward pass:
 - o we want dL/dx , therefore: dL/dy and $dy/dx \rightarrow dL/dx = dL/dy * dy/dx$
- Backpropagation includes multiplication with high/low numbers frequently
- Too high: exploding gradient, too low: vanishing gradient, can be influenced by η
- Backpropagation is no training algorithm, just computes a gradient
- Activation functions (last activation function produces outcome/prediction)
 - o Sign: normalizes input, but gradient vanishes almost everywhere -> not good for gradient descent
 - $f(x) = \begin{cases} +1 & \text{for } x \geq 0 \\ -1 & \text{for } x < 0 \end{cases} \quad f'(x) = 2\delta(x)$
 - o Sigmoid: normalizes output, but gradient still eventually vanishes
 - $f(x) = \frac{1}{1+e^{-x}} \quad f'(x) = f(x) * (1 - f(x))$
 - o ReLU (rectified linear unit): less vanishing gradient, but doesn't normalize input
 - $f(x) = \max(0, x) \quad f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{else} \end{cases}$
- Fully connected layer:
 - o $\hat{y} = Wx$ (just a matrix multiplication) $\hat{y} = W_3 W_2 W_1 x$
 - o Gradient with respect to the weights: $\frac{dL}{dW} = \frac{dL}{d\hat{y}} * \frac{d\hat{y}}{dW} = \frac{dL}{d\hat{y}} x^T$ (matrix)
 - o Gradient with respect to the input: $\frac{dL}{dx} = \frac{dL}{d\hat{y}} * \frac{d\hat{y}}{dx} = W^T \frac{dL}{d\hat{y}}$ (vector)
- After the forward pass through all layers, the loss can be computed



Questions

- Name a loss function for multi-class classification in deep learning: Softmax loss or cross entropy loss
- Explain how this loss function works: Combination of softmax activation and cross-entropy loss. Computes a probability distribution between 0 and 1 that sums up to 1.

- How can you check if the derivative implementation of a loss function is correct. By computing the finite differences.
- What does backpropagation do and how does it work? First compute loss (difference between actual and expected output). Then compute partial derivatives to determine the influence of the specific weights on the loss, to adapt these weights.
- Explain the exploding and vanishing gradient problems: by multiplying with high numbers the gradient becomes bigger and bigger -> explodes, low numbers -> vanishes (update on weights gets smaller until loss doesn't optimize any longer)
- Why is the signum function not used in deep learning? Because it leads to the vanishing gradient problem as the derivative is 0 almost everywhere

Loss and optimization

- Classification: Estimate a discrete variable for every input, model a decision boundary to assign a class to the input
- Regression: Estimate a continuous variable for every input, model a function that explains your data

Loss

- Loss functions: combine all M samples & labels, only present during training, calculates the loss as a scalar
- Maximum Likelihood Estimation:
 - o calculates the probability to observe y_m given observation x_m is $p(y_m | x_m)$
 - o $Y = \prod_{m=1}^M p(y_m | x_m)$
 - o $\underset{w}{\text{maximize}} \{ \prod_{m=1}^M p(y_m | x_m, w) \}$
- Negative Log Likelihood: $\underset{w}{\text{minimize}} \{ \sum_{m=1}^M -\ln(p(y_m | x_m, w)) \}$
- L2-Loss
 - o $\frac{1}{2} \sum_{m=1}^M \|y_m - \hat{y}(x_m, w)\|_2^2 = \frac{1}{2} \sum_{m=1}^M (y_m - \hat{y}(x_m, w))^2$
 - o Corresponds to variants of minimizing the expected misclassification probability
 - o Causes slow convergence, because it doesn't heavily penalize misclassified probabilities
 - o Advantageous in situations with extreme label noise
 - o Use L2 Loss for regression
- Bernoulli distribution for classifications with only 2 classes
- Multi-class generalization: Categorical distribution
- Binary cross entropy
 - o $L = -y \ln(f(x)) - (1 - y) \ln(1 - f(x))$
- Cross-entropy loss
 - o $L(w) = - \sum_{m=1}^M \ln(\hat{y}_k(x_m, w)) | y_{k,m} = 1$
 - o Use Cross-entropy loss for classification
 - o Each predicted class probability is compared to the actual class
 - o Desired output is 0 or 1
 - o A score/loss is calculated that penalizes the probability based on how far it is from the actual expected value
 - o The penalty is logarithmic yielding a large score for large differences close to 1 and small differences tending to 0
- Hinge loss
 - o The classification depends on the sign, if they match we classify as correct
 - o $L(w) = \sum_{m=1}^M \max(0, 1 - y_m \hat{y}(x_m, w))$

Optimization

- Subgradients
 - o Generalization of gradients for convex, non-smooth functions
 - o Subgradient algorithm replaces gradient descent algorithm
 - o Subdifferential is the combination of all subgradients at one point
 - o Use subgradients for ReLU and Hinge loss
- SVM (support vector machine) can be incorporated into a neural network (tries to find a classification with a large margin between the classes which is free of objects)
- Gradient descent
 - o Goal: minimize the convex function using iterations
 - o $w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)}, x, y)$
 - o Stepsize is defined by the learning rate η
 - o Guaranteed to converge to a local minimum
- Batch gradient descent
 - o Preferred option for convex problem
 - o Sum over all examples for each update
 - o Problem: result is non convex, memory limitations
- Stochastic gradient descent

- Does not necessarily increase for every iteration
 - Inefficient because of transfer latency to GPU
 - Uses only 1 sample per gradient
- Mini-batch SGD
 - Sums up over lower number of examples based on batch size
 - Small batches offer regularization effect -> need smaller η
 - Regains efficiency
- Learning rate η
 - Too small: long training time
 - Too large: miss optima
- Momentum μ
 - Parameter update based on current and past gradients
 - Lower the computation time by introducing the concept of experience
 - Average the decay, when the decay increases the decision making decreases
 - $v^{(k)} = \mu v^{(k-1)} - \eta \nabla L(w^{(k)}) \quad w^{(k+1)} = w^{(k)} + v^{(k)}$
- Nesterov Momentum
 - Look ahead gradient step
 - $v^{(k)} = \mu v^{(k-1)} - \eta \nabla L(w^{(k)}) \quad w^{(k+1)} = w^{(k)} + \mu v^{(k)} + (1 - \mu)v^{(k)}$
- Adaptive Gradient (AdaGrad): Adaptation based on all past squared gradients (individual learning rates)
- RMSProp: like AdaGrad but aggressive decrease is fixed
- Adadelta: needs no learning rate
- Adam: Adaptive Moment Estimation
 - Very robust / well behaved
 - Extension to gradient descent
 - Computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients
 - Individual learning rates
 - Loss curves hard to interpret
- SGD + Nesterov Momentum + learning rate decay
 - Often converges most reliable
 - Learning rate decay needs to be adjusted

$$\begin{aligned}
 \mathbf{g}^{(k)} &= \nabla L(\mathbf{w}^{(k)}) \\
 \mathbf{v}^{(k)} &= \mu \mathbf{v}^{(k-1)} + (1 - \mu) \mathbf{g}^{(k)} \\
 \mathbf{r}^{(k)} &= \rho \mathbf{r}^{(k-1)} + (1 - \rho) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)} \\
 \text{Bias correction: } \hat{\mathbf{v}}^{(k)} &= \frac{\mathbf{v}^{(k)}}{1 - \mu^k} \quad \hat{\mathbf{r}}^{(k)} = \frac{\mathbf{r}^{(k)}}{1 - \rho^k} \\
 \mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} - \eta \frac{\hat{\mathbf{v}}^{(k)}}{\sqrt{\hat{\mathbf{r}}^{(k)} + \epsilon}}
 \end{aligned}$$

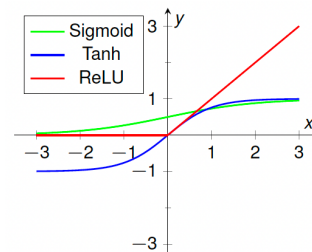
Questions

- What are our standard loss function for classification and regression: L2-loss for regression, cross-entropy loss for classification
- What assumptions do our standard loss functions imply? They can both be derived as Maximum Likelihood Estimators from strict probabilistic assumptions, functions are smooth (differentiable)
- What is a subdifferential at point x_0 ? A set of subgradients at x_0 .
- How can we optimize a non-smooth convex function? With subgradients, they are a generalization of gradients for gradient descent algorithm for convex, non-smooth functions
- What if somebody tells you, to use an SVM because it is superior? It is up to a multiplicative constant the same thing as using hinge loss. If you choose the right loss function you can also incorporate a SVM into your deep learning framework.
- What is Nesterov momentum? We add a momentum before computing the gradient to approximate the next set of parameters. Nesterov includes a look ahead step.
- Describe Adam: Computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients.

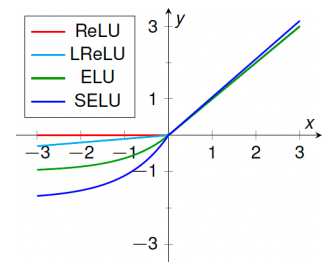
Activation functions and convolutional neural networks

Activation functions

- Linear activation function
 - $f(x) = ax \quad f'(x) = a$
 - Simple, renders the optimization problem convex
 - Does not introduce non-linearity -> not used in DL
- Sigmoid activation function
 - $f(x) = \frac{1}{1 + e^{-x}} \quad f'(x) = f(x) * (1 - f(x))$
 - Probabilistic output
 - But saturates for $x \ll 0$ and $x \gg 0$
 - Not zero-centered
- Zero-centering: output always +, gradient of weights either always + or -, mean $\mu = 0$ of input distribution will be shifted to $\mu > 0$ -> covariate shift of successive layers, layers have to adapt to shifting distribution
- Tanh activation function
 - $f(x) = \tanh(x) \quad f'(x) = 1 - f(x)^2$
 - Zero-centered
 - But still saturates, causing vanishing gradients



- Rectified Linear Units (ReLU)
 - o $f(x) = \max(0, x)$ $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{else} \end{cases}$
 - o Good generalization due to piece-wise linearity
 - o speed up during learning
 - o no vanishing gradient problem
 - o but not zero-centered, doesn't normalize input
 - o enable training of deep supervised neural networks without unsupervised pre-training
- Dying ReLUs: for negative values it is zero, hence there are no more updates
- Leaky ReLU: αx instead of zero for negative x with $\alpha = 0.01$, fixes dying ReLU problem
- ELU: $f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$, no vanishing gradient, reduces shift in activations
- SELU: scales exponential linear unit, parameter λ introduces self-normalization
- Finding the best activation function is a difficult, expensive optimization problem
- Characteristics of good activation functions:
 - o Have almost linear areas to prevent vanishing gradients
 - o Have saturating areas to provide non-linearity
 - o Should be monotonic



Convolutional neural networks

- Pixels are a bad representation/features from a ML view, because they are highly correlated, scale dependent, have intensity variations
- Learn a better representation before the classification
- Local connectivity -> filters
- Use same filters over the whole image -> translational equivariance
- Hierarchy of filters working on different scales + learning = convolutional neural networks
- Building blocks: (1-3 for feature learning)
 - o Convolutional layer: feature extraction
 - o Activation function: Nonlinearity
 - o Pooling layer: compress and aggregate the information, save parameters
 - o Last layer: fully-connected for classification (can be replaced with flatten and 1x1 convolution)
- Local connectivity: features that are important at one location are likely important anywhere in the image -> use the same weights everywhere
- Convolution with trainable filters
- Cross-correlation = convolution with a flipped kernel
- Same / zero padding: input and output have the same size
- Valid / no padding: output is smaller than the input
- Stack multiple filters to get a trainable filter bank
- Convolution is independent of image size
- Much more training data for one weight
- Strided convolution: skip some positions with stride s , reduces the output by a factor of s -> convolution + subsampling
- Dilated/Atrous convolutions: skip certain pixels, wider receptive field with less parameters/weights
- Convolution with 1x1 filter is same as fully connected
 - o Flattens the input to have H channels (= number of filters)
 - o Simple and efficient method to decrease the size of a network
 - o Learns dimensionality reduction, can reduce redundancy in your feature maps
 - o Can replace fully connected layer, enables arbitrary input sizes in combination with global average pooling
- Inception model: combine different filters / neighborhoods / pooling

Pooling

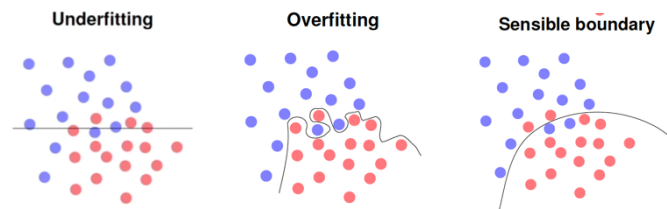
- Fuses information of input across spatial dimensions
- Decreases number of parameters
- Reduces computational costs and overfitting
- Assumptions: features are hierarchically structured, creates summaries of regions, provides translational invariance, exact location of a feature is not important
- Stride size usually equals the neighborhood size (ex: 2x2)
- Max pooling
 - o propagates maximum value in a neighborhood to next layer, adds non-linearity
 - o Only one value contributes to error, which is propagated along the path of the maximum value
- Average pooling
 - o Propagate average of the neighborhood
 - o Backward pass: error is shared to equal parts
- Additional pooling strategies: fractional max pooling, stochastic pooling, generalized pooling, ...
- Alternative to pooling: strided convolution, which allows for trainable downsampling



Questions

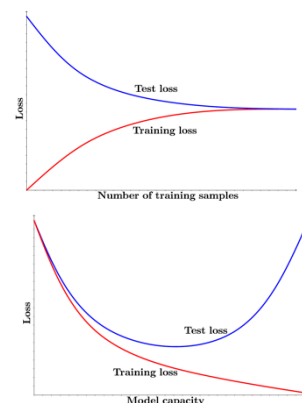
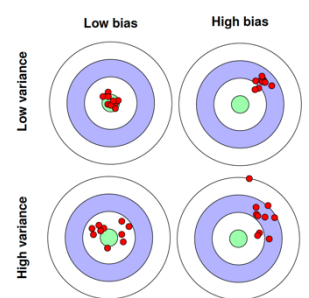
- Name 5 activation functions: Linear, sigmoid, tanh, ReLU, LReLU
- Discuss those 5 activation functions:
 - o Linear: + simple, - but doesn't introduce nonlinearity and therefore renders a convex optimization problem
 - o Sigmoid: + Nonlinear, Close to biological model but differentiable and renders probabilistic output, - saturates for $x \ll 0$ and $x \gg 0$ (vanishing gradients), not zero centered (internal covariate shift)
 - o Tanh: + nonlinear, differentiable (shifted version of sigmoid) and zero centered, - still saturates and therefore has the vanishing gradient problem
 - o ReLU: + good generalization due to piece wise linearity, speed up during learning, no vanishing gradient, - not zero centered, dying ReLU problem when all activations are < 0
 - o LReLU: + fixes dying ReLU problem, apart from that same as ReLU
- What is the zero-centering problem? Non zero-centered activation functions cause a shift of the input distribution in the following layers, so the layers constantly have to adapt to the shifting distribution -> internal covariate shift
- Why does ReLU as activation function perform much better than sigmoid/tanh in a large number of tasks? Because for $x > 0$ ReLU does not saturate but is instead linear and has a derivative of 1. Therefore there is no vanishing gradient problem like for sigmoid and tanh which saturate for small and big x and therefore have a derivative of 0 in those places.
- Why are convolutional networks well suited for image and audio processing? Because CNNs create a better feature representation for the input.
- Write down a mathematical description of strided convolution. $(f * g) = \sum_{x=0}^{\text{output size} - \text{stride}} f(\tau)g(\text{stride} * x - \tau)d\tau$
- What is the connection between 1x1 convolutions and fully connected layers? If we flatten the input, 1x1 convolutions are fully connected layers.
- How would you implement a classifier which operates on image patches? I would use CNN layers with pooling and ReLU in the beginning to get a feature representation and reduce the dimensionality, for the last layers I would use a fully connected layer with softmax to get a feature vector.
- What is a pooling layer? In pooling layers the dimensionality is reduced by iterating over the input with a kernel and within the kernel size (choosing the maximum) and only using this value for the output
- Why do we use pooling layers? Dimensionality reduction, reduces computational costs and overfitting
- On what data would CNNs probably perform bad? Sequential data like language (text)

Regularization:



- Noisy data -> classes are not easily separable
- Underfitting: model with low capacity = bad fit for describing the separation of the classes
- Overfitting: model with very high capacity = decision boundaries that are not reasonable
- Sensible boundary: compromise between observed data and actual distribution
- Bias variance decomposition: the loss of an ideal function can be composed into the Bias² + the variance + an irreducible error:

$$\mathbb{E}_D(\ell(\mathbf{x})) = \underbrace{\mathbb{E}_D[\hat{\mathbf{y}}_D] - h(\mathbf{x})}_{\text{Bias}^2}^2 + \underbrace{\mathbb{E}_D[(\hat{\mathbf{y}}_D - \mathbb{E}_D[\hat{\mathbf{y}}_D])^2]}_{\text{Variance}} + \underbrace{\sigma_\epsilon^2}_{\text{Irreducible Error}}$$
- Measures how far we are off the ground truth
- The training data set is limited in size
- Bias-Variance tradeoff: if you reduce the bias (with a flexible model), the variance increases
- Simultaneously optimizing bias and variance is impossible
- Bias = prediction error of our model in the training data set
- High variance: overfitting, high bias: underfitting
- Bias and variance together are the model capacity
- The model capacity determines the variety of functions it can approximate and is related to the number of parameters
- Vapnik-Chervonenkis (VC) dimension computes the model capacity (Based on counting how many points can be separated by a model)
- The bias decreases when you increase the model capacity
- Training loss increases with the number of parameters and test loss decreases.
- Small datasets have a high test loss and are not representative
- Optimize variance by using more training data
- If you increase the model capacity the loss goes down (both training and data)
- > A higher capacity can memorize more of the training data set
- If you increase it too much you get an overfit and the test loss increases again.



- Regularization should reduce overfitting (by enforcing prior knowledge)
- Use validation data in order to produce a surrogate of the test loss
- Methods to enforce prior knowledge:
 - o Data augmentation
 - o Adapt architecture
 - o Adapt training process
 - o Preprocessing
- Actual regularizers: constrain the solutions to equality or inequality constraints

Classical regularization methods

- Never use test set for training
- Split of part of the training set as a validation set
- Validation set to observe at what point we have the minimal error in our validation set, use this point as a stopping criterion
- is used to give an estimate of model skill while tuning model's hyperparameters, to compare or select between final models
- using data that was part of the training would give a very high score that is biased. Using the validation set results in an unbiased score
- Data Augmentation: artificially enlarge data set with transformations
- Random spatial transformations: affine transformations, elastic transformations
- Pixel transformations: changing resolution, random noise, changing pixel distribution (color, brightness)
- Regularize the loss function -> maximum a posteriori approximation
- Bayesian approach: $p(w|Y, X) = \frac{p(Y|X, w) * p(X, w)}{p(Y, X)} = \frac{p(Y|X, w) * p(X) * p(w)}{p(Y|X) * p(X)}$
- MAP(w) := $\underset{w}{\text{maximize}} \{ \underbrace{p(Y|X, w)}_{\text{MLE estimator}} \underbrace{p(w)}_{\text{Prior}} \}$
- Solve with maximum likelihood estimation
- Use prior knowledge like sparcities
- The maximization of the maximum likelihood estimates results in a minimization of the negative log likelihood
- Loss function changes: $\tilde{L}(w, X, Y) = L(w, X, Y) + \lambda \|w\|_2^2$ -> enforces a minimum L2 Norm
- Changes the loss function to penalize certain values of the weights
- $\tilde{L}$ is like L2 loss or cross entropy loss plus hyperparameter λ which adds some constraints on the weights
- λ determines how much we regularize our model
- Use cross validation to determine λ
- Normalization with the L2 norm leads to smaller weights, since large weights are penalized quadratically!
- Backpropagation of augmented loss: $w^{(k+1)} = \underbrace{(1 - \eta\lambda)w^{(k)}}_{\text{shrinkage}} - \eta \frac{\partial L}{\partial w^{(k)}}$
- Implementation: follow the loss of the gradient, use shrinkage in the weight update
- Shrinkage step computes the additional L2 regularization
- Shrinkage function can also be called weight decay
- For different regularizers the shrinkage function changes
- Regularization is not good for training loss, because it reduces overfitting
- We trade increased bias for reduced variance
- With L1 norm: $\tilde{L}(w, X, Y) = L(w, X, Y) + \lambda \|w\|_1$
- Backpropagation of augmented loss: $w^{(k+1)} = \underbrace{w^{(k)} - \eta\lambda \text{sign}(w^{(k)})}_{\text{shrinkage}} - \eta \frac{\partial L}{\partial w^{(k)}}$
- Normalization with L1 norm causes sparse weights, because some weights are close to zero
- Use L1 norm if you want networks with few connections
- Norm constraints (sets a limit on the norm of the weights) prohibit exploding gradients
- We can have a constraint and a λ individual for every layer
- Instead of weights, activations can be constraint as well

Regularization

- Normalization: Subtract mean, then normalize variance -> data set is in the expected range regarding the input
- Only use training data to calculate normalization
- Batch normalization is normalization within the network
- Introduce new layers with the parameters γ and β , which are used to rescale the output of the layer
- Compute mean and standard deviation of mini-batch
- Use it to normalize the activations of the input -> result: zero mean and a unit standard deviation
- $\tilde{x}_i = \frac{x_i - \mu_{B,i}}{\sqrt{\sigma_{B,i}^2 + \epsilon}}$
- Scale the output of the layer with the trainable weight β and γ
- $\hat{y}_i = \gamma_i \tilde{x}_i + \beta_i$
- For test time you compute the mean and standard deviation for the entire training set and keep it constant
- Batch Normalization reduces Internal covariate shift

- ReLU is not zero-centered, initialization and input distribution might not be normalized -> input distribution shifts over time -> layers constantly have to adapt which leads to slow learning
- But there is very little evidence
- Experience shows that batch normalization is effective
- BN smoothes loss landscape and improves stability with respect to hyperparameters, initialization and convergence

Self normalizing neural networks

- Addresses the stability problem of SGD (stochastic gradient descent)
- Keyelement: SeLU (scaled exponential linear unit) + specific weight initialization
- Function: $f(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{else} \end{cases}$
- Choose λ and α in a way that mean and std stay near 0 and 1 through the network -> stable activations and stable training
- Requires special dropout: instead of zero drop to $-\lambda \cdot \alpha$
- Use an affine transformation to restore variance and mean
- SNN concept resembles batch normalization and leads to an increased stability

Dropout

- All elements during training depend on each other
- Break the correlation between features of independent features
- Randomly kill some neurons and set them to zero with probability $1 - p$
- Multiply all activations with p during test time (compensates for a too high activation)
- Breaks internal correlations, weights get drawn in a different direction
- Reduced the problems of overfitting
- Dropconnect: kill connections instead of neurons
- Less efficient, because implementation requires masking

Initialization

- Initialization doesn't matter for convex functions, because you will always be able to find the global minimum
- Non convex functions can have several local minima -> different initializations result in different minima
- Initialize Bias with zero, if ReLU is used initialize with a small positive constant 0,1 because of the dying ReLU issue
- Weights need to be random to break the symmetry
- Do not initialize with zero, because then the gradient is zero
- The variance of the weights influences the stability of the learning process
- Small uniform / Gaussian values work
- Variance depends on number of inputs for the layer
- $\text{Var}(Y) = \underbrace{N \text{Var}(Wn)}_{\text{scales Var}} \text{Var}(X_n)$ more weights = higher variance
- Xavier initialization: calibrate the variances for the forward pass, initialize with a zero-mean Gaussian $N(0, \sigma)$
- Forward pass: $\sigma^2 = \frac{1}{fan_in}$ with fan_in = input dimension of the weights
- Backward pass: $\sigma^2 = \frac{1}{fan_out}$ with fan_out = output dimension of the weights
- $\sigma = \sqrt{\frac{2}{fan_out + fan_in}}$
- He initialization: used for ReLU, identifies the assumption of linear neurons as a problem
- $\sigma = \sqrt{\frac{2}{fan_in}}$
- Conventional initial choices:
 - o L2 regularization
 - o Dropout with $p=0,5$ for FCN and selectively in CNN
 - o Mean subtraction
 - o Batch normalization
 - o He initialization

Transfer learning

- Used when you only have little data available
- Reuse models trained for example on ImageNet
 - o For a different task on the same data
 - o On different data for the same task
 - o On different data for a different task
- Convolutional layers extract features
- Less task-specific features in earlier layers -> worth transferring
- Cut the network at some depth in the feature extraction part
- Fix the learning rate to zero for those extracted parts $\eta = 0$ or do fine tuning
- Example: Skin cancer classification
- Replace the training part of an existing network
- Transfer between modalities is also possible
- Transfer learning is typically a good idea

Multi-Task Learning MTL

- Learning simultaneously can provide better results of the shared underlying concepts than relearning
- Train a network simultaneously on multiple related tasks
- Adapt the loss function to assess performance for multiple tasks
- Introduces inductive bias
- Reduces the risk of overfitting on one particular tasks -> generalizes better
- We have some shared input layers: feature extraction layers or representation layers
- Split at some point into task specific layers to evaluate on the different tasks
- Multi-task learning of N tasks reduces chance of overfitting by an order of N
- This is hard sharing
- Soft parameter sharing introduces an additional loss
- You constrain the different activations to be similar
- Each model has its own parameters, but we link them together to perform similar yet different extraction steps in the constraint layers. For example with an L2 norm
- All of the tasks should have their own purpose
- Auxiliary tasks can make the network more stable
- Auxiliary task can help steer the training in a certain direction and includes prior knowledge

Questions

- What is the bias-variance tradeoff? If you reduce the bias, the variance increases. Simultaneously optimizing both is impossible.
- What is model capacity? Describes the bias and variance together. Determines the variety of functions it can approximate and is related to the number of parameters
- Describe three techniques to address overfitting in a neural network. Validation set (define stopping criterion), Data Augmentation (enlarge data set with transformations), Dropout (break correlations between independent features)
- Why do we often need a validation set? Use the point with the minimal error as stopping criterion, compare or select between final models.
- Can we optimize the hyperparameter λ by gradient descent on the training set? We can optimize it with cross validation.
- What connects the covariate shift problem? The ReLU is not zero-centered, therefore a covariate shift arises. Effect gets larger with every iteration.
- How can we address the covariate shift problem? With Batch Normalization we can normalize every batch, so that the data is zero-centered again.
- Which problems do current initialization schemes try to address? For non-convex functions multiple local minima exist. But we need the global minimum
- What is transfer learning? Re use part of an already trained network.

Common Practices

- Never use test data for training!

Training strategies

- Check gradients before training (by comparing analytical and numerical gradient) with only a few datapoints
- Check again after training for a short period of time, with and without regularization, turn off data augmentation and dropout
- Check initialization and loss before training
- Check whether the architecture is in general capable to learn the task
 - o Take a small subset and try to overfit the network to get zero loss
 - o If the network cannot overfit, the network isn't learning properly / not suitable for the task, or the model is too small (increase the number of parameters) or bug in implementation
- Monitor loss function
 - o Check learning rate and identify large jumps in the learning curve
 - o Very noisy curves can be associated with too small mini batches -> increase batch size
- Monitor validation loss using a validation data set
 - o If training and validation loss diverge: overfitting -> increase regularization or early stopping
 - o If they are close but high: underfitting -> decrease regularization / increase model capacity
- Monitor weights and activations
 - o Relative magnitude of weight update should be in a sensible range ($1e-3$)
 - o Convolutional layers: check filters of first few layers, should be smooth and regular filters
 - o Check for very large or saturated activations (dying ReLUs)

Optimization and Learning rate

- Choosing an optimizer
 - o Batch gradient descent: requires large memory, too slow, too few updates
 - o SGD: usual choice, loss function and gradient become very noisy if only one/few samples are used
 - o SGD + mini-batch: frequent, more stable updates, gradient noisy enough to escape local minima, adapting mini-batch size yields smoother/more noisy gradient

- Addition of momentum prevents oscillations and speeds up optimization
 - Recommendation: start with Mini-batch SGD + momentum, use Adam for faster convergence speed
- Learning rate has a large impact on the successful training of a network
- We want an adaptive learning rate
- Annealing the learning rate = learning rate decay
 - Decay strategies: stepwise decay (most common), exponential decay, $1/t$ -decay

Architecture selection and hyperparameter optimization

- Start with simple architectures and loss functions and try well-known models first and foremost
- Hyperparameter search
 - Search for hyperparameters using a log scale
 - Use random search instead of grid search (easier to implement, better exploration of parameters that have strong influence on the result)
 - Optimize on a coarse to fine scale

Ensembling

- Combine the results of different learning algorithms to get better overall results
- Using several classifiers yields better results, if they are independent
- Produce N independent classifiers/regressors and combine their predictions by majority / averaging
- Use different local minima and combine the models across the optimization process
- Use different model checkpoints and different methods
- Easy performance boost if you need just a bit more

Class imbalance

- Problem: one class (the interesting one) is not very frequent
- Sometimes classifying everything to the most common class gets you a high accuracy but is not the goal
- Undersampling: In each iteration only take a subset of the overrepresented class (throw away data points), to make the appearance of every class equally often
 - Disadvantage: not all available data is used for training and can lead to underfitting
 - Underfitting can be reduced by taking a different subset after each epoch
- Oversampling: use sample from underrepresented class multiple times
 - Disadvantage: can lead to overfitting towards less frequently seen samples
 - Overfitting can be reduced by data augmentation
- Also possible: combination of over- and undersampling
- Instead of fixing the data, the loss function can be adapted to be stable with respect to the class imbalance, with factor $-w_k$
- Instead of class frequency, weights can be adapted with regards to other considerations

Evaluation

- Evaluate generalization performance on previously unseen data: the test set
- Data is labeled by humans, there can be errors
- Mistakes by the classifier are less severe if the same classes are confused by humans
- Performance measures: sensitivity and specificity and precision
 - Top-K error: True class label is not in the K classes with the highest prediction score
- K-fold cross validation for very little data
 - Use k-1 folds as training data, test on fold k, repeat k times
- Comparing classifiers: is there a significant difference between the classifiers (with t-test)
- Bonferroni correction: use multiple comparisons

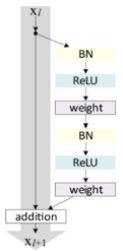
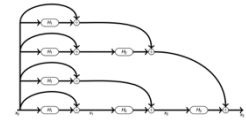
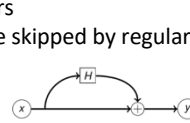
Architectures

- Early Architectures: LeNet-5, AlexNet, Network in Network, VGG Network, GoogleNet,
- Later Architectures: Inception-ResNet, ResNeXt, ResNet-of-ResNets, SENet
- Advantages of deeper networks
 - More layers = more paths (4 layers, 2 neurons = 2^4 paths)
 - Deeper networks seem to somehow be able to reuse information from the previous layer
 - Increasingly abstract features possible
- Inception
 - You don't have to choose for one filter/convolution. Just do all of them and concatenate the result
 - Because it is highly computational expensive only take the bottleneck layer
- Bottleneck layer (1x1)
 - Features are correlated, redundancy can be removed by 1x1 filter: reduce the channels
 - 3x3 & 5x5 convolution
 - Enables the network to learn various spatial patterns at different scales as a result of the varying conv filter sizes
- Label smoothing
 - For softmax to predict exactly 0/1, the activations approach ∞ . ->Continue to learn larger and larger weights, making more extreme predictions
 - Use weight decay and/or label-smoothing: $q'(k|x) = (1 - \epsilon)\delta_{k,y} + \epsilon u(k)$

- Prevent hard probabilities without discouraging correct classification
- Helps with noisy labels

Residual Network

- Deeper models tend to have higher training & test error than shallower models (not just caused by overfitting)
 - Reasons: Vanishing gradient, internal co-variate shift, degradation problem
- ResNet: skip connections: connects activations to further layers
- If any layer hurts the performance of the architecture it will be skipped by regularization
- Residual unit: $H(x) = F(x) - x \Leftrightarrow F(x) = H(x) + x$
- A Residual Block is a residual unit that skips several steps
- Residual network contains several residual blocks
- Bottleneck Residual Blocks with 1x1 convolutions enable deeper networks and larger input image sizes
- Ensemble view combines several residual units:
 - $x_3 = [x_0 + H_1(x_0) + H_2(x_0 + H_1(x_0))] + H_3(x_0 + H_1(x_0) + H_2(x_0 + H_1(x_0)))$
- At layer level: 2^n paths (classical feed-forward network: one single path)
- At neuron layer: many different paths of varying (ffn: same) length going through different subsets of layers
- Residual blocks do not compute entirely new representations, they instead iteratively refine their input representations
- Residual networks allow removal of connections without significant drop in performance
- Stochastic depth:
 - Layer-wise dropout, drop random layers and bypass with identity
 - Networks are shorter during training -> decreased training time
- Wide residual networks
 - Decrease depth, increase width of ResNet blocks
 - Use dropout in residual blocks
 - Power not from depth but from residual connections
- DenseNets (Densely Connected Convolutional networks)
 - Layer input: feature-maps of all preceding layers
 - Feature propagation, feature reuse
- Squeeze and excitation networks (SENet)
 - Idea: Add trainable module that allows rescaling of channels depending on input
 - Squeeze: compress each channel into one value (global avg. pooling)
 - Excitation: FC layers & sigmoid to achieve scaling vector -> compare to gating in LSTMs
 - Scale: scale input feature maps



Learning Architectures

- Goal: Self-developing network structures
- Reinforcement learning: Recurrent neural network to generate model descriptions of networks
- Train RNN with reinforcement learning to maximize expected accuracy

Conclusion

- 1x1 filters to reduce parameters and add regularization
- Inception layers
- Residual connections
- New architectures can be learned
- Rise of deeper models (from 5 layers to more than 1000)
- Often a smaller net is sufficient
- Dependent on amount of training data
- Deep vs wide layers

Questions

- What are the advantages of deeper models in comparison to shallow networks? More layers = more paths, able to reuse information from the previous layer, increasingly abstract features possible.
- Why can we say that residual networks learn an ensemble of shallow networks? Consist of many ensembled residual units, we can break down the structure in kind of recursion.
- How does a bottleneck layer work? Features are correlated, redundancy can be removed by 1x1 filters to reduce the channels.
- What is the standard inception module and how can it be improved? Parallel filter combinations (combination of 1x1, 3x3, 5x5, max-pooling). V2: efficient grid size reduction + average pooling + softmax. V3: RMSProb, batch normalization, label smoothing regularization. The standard inception module is composed of multiple parallel convolutional layers of different filter sizes, followed by a max pooling layer, and a concatenation layer that combines the output from each parallel branch.

Recurrent neural networks

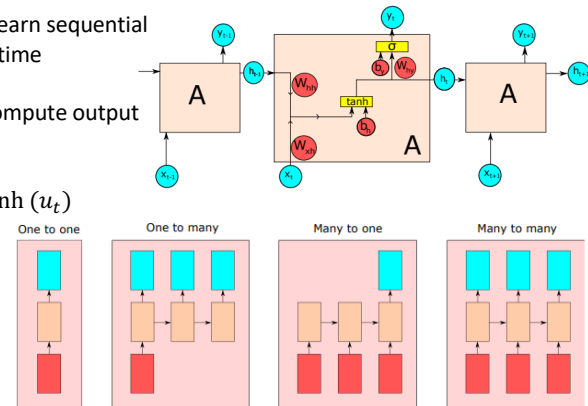
Motivation

- Feedforward neural networks do not take temporal context into consideration
- Examples for sequential or time dependent signals: Speech/Music, video, sensor data

- Example problem: you can't translate a sentence only knowing one word, but you also can't feed the whole sequence to a big network (inefficient memory usage, difficult to train, not real time)
- Solution: model sequential behavior within architecture: RNNs

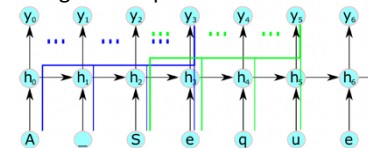
Simple recurrent neural networks

- Properties of RNN: can model loops, model memory and experience, learn sequential relationships, provide continuous predictions as data comes in -> real time
- X_t -> current input, additionally hidden input h_{t-1}
- Feedback loop: information from present and recent past is used to compute output
- Tanh: Combination of previous state and current input
- σ : Additional non-linearity for output
- Update hidden state: $h_t = \tanh(W_{hh} * h_{t-1} + W_{xh} * x_t + b_h) = \tanh(u_t)$
- Output formula: $y_t = \sigma(W_{hy} * h_t + b_y) = \sigma(o_t)$
- Examples of different architectures:
 - o One to one -> Image classification (classical feed forward)
 - o one to many -> Image captioning
 - o many to one -> Sentiment analysis
 - o many to many -> video classification
- Similar to CNNs, stack multiple units for deep RNNs



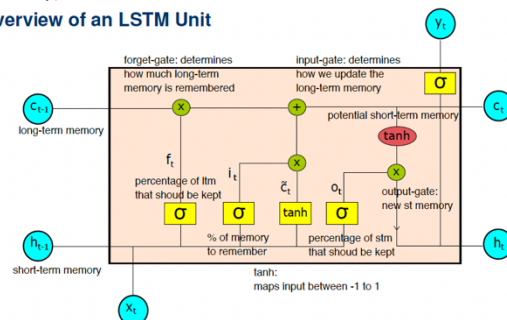
Training RNNs

- Backpropagation through time (BPTT)
 - o Compute forward pass for the full sequence -> loss (cross-entropy loss: $L(\hat{y}, y) = \sum_{t=1}^T L(\hat{y}_t, y_t)$)
 - o Compute backward pass through full sequence to get gradients -> weight update
 - o Compute gradient of the loss function: $\nabla \theta = \theta [\nabla W_{xh}, \nabla W_{hh}, \nabla W_{hy}, \nabla b_h, \nabla b_y, \nabla h_0]$
 - o Update parameters using a learning rate η : $\theta = \theta - \eta \nabla \theta$
 - o Iterate backwards through network from $t=T$ to $t=1$
 - o $\nabla o_t = \sigma'(o_t) * \frac{dL}{dy_t}(\hat{y}_t, y_t)$ $\nabla W_{hy,t} = \nabla o_t h_t^T$ $\nabla b_{y,t} = \nabla o_t$
 - o $\nabla h_t = (\frac{dh_{t+1}}{dh_t})^T * \nabla h_{t+1} + (\frac{do_t}{dh_t})^T * \nabla o_t = W_{hh}^T * \tanh'(W_{hh} h_t + W_{xh} x_{t+1} + b_h) * \nabla h_{t+1} + W_{hy}^T \nabla o_t$
 - o $\nabla W_{hh,t} = \nabla h_t * \tanh'(u_t) * h_{t-1}^T$ $\nabla W_{xh,t} = \nabla h_t * \tanh'(u_t) * x_t^T$ $\nabla b_{h,t} = \nabla h_t * \tanh'(u_t)$
 - o For each gradient simply sum over all time-steps
- Normal BPTT: One update requires backpropagation through a complete sequence, single parameter update is very expensive
- Naïve BPTT: Split long sequence into batches of smaller parts, but blind to long term dependencies
- Truncated BPTT (TBPTT)
 - o Keep processing sequences as a whole
 - o Every k_1 time steps run BPTT for k_2 time steps
 - o Hidden states are still exposed to many time steps
 - o Works for short-term dependencies, but not for long-term
- The Long-term dependency problem with basic RNNs: Layers and time steps of deep RNNs are related through multiplication: Gradients prone to vanishing or exploding
- Short-term dependencies hide long-term dependencies due to exponentially small gradients



LSTM (long short term memory)

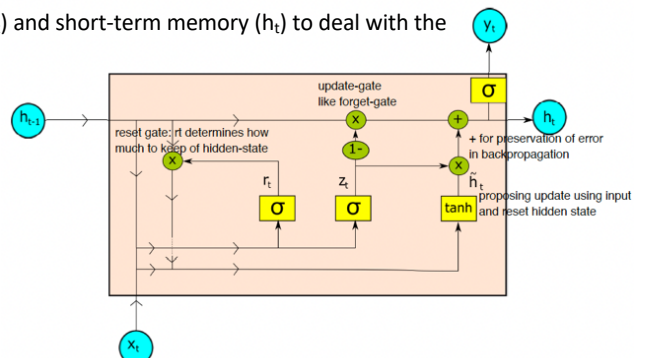
Overview of an LSTM Unit



- Main concept: have separate paths for long-term memory (c_t) and short-term memory (h_t) to deal with the vanishing / exploding gradient problem

GRU (gated recurrent unit)

- More control over hidden state / memory using gates
- Main difference: no additional cell state
- Reset-gate: influence of previous hidden-state
- Update-gate: influence of newly computed update



Differences

LSTM	GRU
Seperate hidden and cell state	Combined hidden and cell state
Controlled exposure of memory content through output gate	Full exposure of memory content without control
Independent input and forget gate: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ → New memory content independent of current memory	Common update gate: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ → New memory content depends on current memory

- GRU clearly outperforms regular recurrent unit
- Comparison between GRU and LSTM not conclusive, similar performance

Sampling strategies for RNNs

- RNN actually computes probability distribution of the next element
- Question: How do we sample this distribution?
- Greedy search: At each point, pick the most likely element
 - o Drawback: No lookahead possible
 - o Tends to repeat sequences of frequent words
- Beam search: select k most likely elements
 - o Out of all possible sequences that have one of these elements as a prefix, take k most probable ones, then iterate until end of sequence
 - o Can generate k sequences in one go: usually better than greedy search
- Random sampling: sample next word according to output probability distribution
 - o If word has output probability 0.8, it is sampled 8 out of 10 times as the next word
 - o Creates very diverse results, but can look to random
 - o To reduce randomness, increase/decrease probability of probable/less probable words: temperature sampling

Questions

- What is the strength of RNNs compared to feed-forward networks? RNNs model sequential behavior within the architecture while feed forward networks don't take the temporal context into consideration
- What role does the hidden state play in RNNs? The hidden state stores information of the past which then influences the calculation of the output y.
- How do you train RNNs? What are the challenges? RNNs are trained with backpropagation through time. The challenges are calculating the gradients for all instances of the RNN cell for all time points. Therefore one parameter update is very expensive.
- What is the main idea behind LSTMs and what is the main difference to simple RNN units? The main idea is introduction of gates that control writing and accessing memory in additional cell state. The main differences are that the LSTM has two states (hidden an cell) as opposed to one int the simple RNN. Also the LSTM has gates which the simple RNN doesn't have.
- What are the differences between LSTMs and GRUs? LSTM has two states, GRU has one. Therefore with LSTMs there is more control over memory possible, because the new memory content is independent of the current memory content. With GRUs no control over the memory content is possible, because the new memory content is dependent on the old memory content.
- In which scenarios would LSTMs be beneficial compared to GRUs? Similar performance. Long-term dependencies.
- Name three applications where many-to-one and one-to-many RNNs would be beneficial. Many-to-one: Sentiment analysis, text classification. One-to-many: Image captioning.

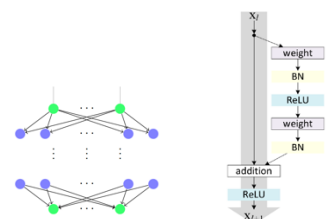
Visualization and Attention mechanisms

Motivation

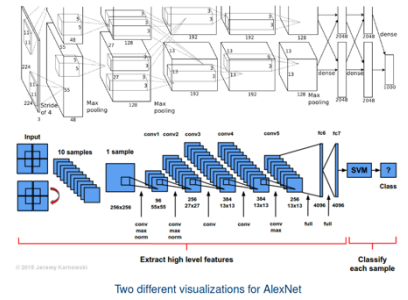
- To communicate the inner-workings of a network and architectures to other people
- To identify issues during training (not converging, dying ReLUs)
- To identify faulty training or test data
- To understand how, why and what networks learn
- Types of visualization:
 - o Architecture
 - o Training
 - o Learned parameters/weights

Network Architecture Visualization

- Communicate effectively what is important about the specific type of neural network
- Priors imposed by the architecture may be most important factor for good performance
- Mostly done with graph based structures with different degrees of granularity
- Node-link diagrams: Neurons = nodes, (weighted) connections = edges
 - o Detailed representation, focus on connectivity
 - o Only for small (sub-)networks, building blocks
 - o Usefull if you want to show the difference between a convolutional layer or a fully connected layer

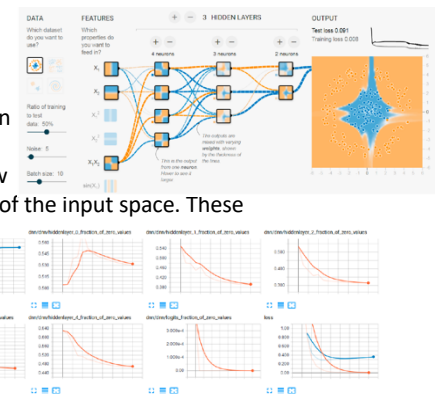


- Different variants with explicit weights, recurrent connections...
- Block diagrams: Layer = solid block, single connection between layers
 - Block: mathematical operation/layer
 - Arrows: direction of the flow of the data
 - Blocks can have different granularity – often hierarchical descriptions
 - Can be used in combination with node-link diagrams to make sure which block does what kind of operations
 - Textual description for the hyperparameters (filter size, number of filters, activations, feature maps) in the caption or as small notes is important
 - Focus on the concept of the architecture
- Other techniques:
 - Graphcore Poplar: fancy graph visualization (very complex)



Visualization of the training

- Contains information on the input data (images, text), parameters (weights, biases), hidden layer data (activations, hidden states), output data (classification, loss curves)
- Track what happens during the training for debugging and improving the model design
- Use TensorFlow / TensorBoard to monitor the training
- Visualizes the activations of a 2D input, see during the training over the iterations how the representation of the different layers change. First layers create binary partitions of the input space. These different partitions can then be assembled to form a more complex shape
- For large problems TensorBoard is better (monitors the actual progress during the training)
- You can see how the loss and other parts are behaving



Visualization of Parameters

- Helps to investigate unexpected/unintuitive behavior, to identify confounds, explain why a network works or why not, increase confidence in the predictions, understand/investigate limitations (adversarial examples)
- Goal is to understand how the network represents data
- Using the right (representative) data for training is very important
- Confounder (for example microphone characteristic for audio recording) a feature
- ML focuses on the most discriminative feature
- Solution: actively include the information about the confounder and try to suppress it. Then you can remove the bias that is introduced by the confounder
- Examples for confounders: sensors, lighting condition, age, gender of the participants, temperature, ...
- Adversarial example: differs only by specifically optimized added noise
- Introduce a noise pattern onto the image which differs only in one or two gray values in each color channel, but the noise pattern was constructed with knowledge about the network in a way that it tries to increase the output for a certain class maximally by introducing random inputs.
- This additional noise pattern can be designed that it will maximize the activation for the wrong class. Without being recognizable for humans.
- It accumulates over the different layers the small inputs at every pixel, but averaged and added up over the different layers, it allows to introduce a small shift. This small shift over the layers can increase to force the network to go to the wrong decision.
- Adversarial examples are generated to cause a specific mistake
- Can be used to confuse face recognition software
- Toaster sticker will cause a classification of the class toast for several kinds of architectures that have been trained on ImageNet



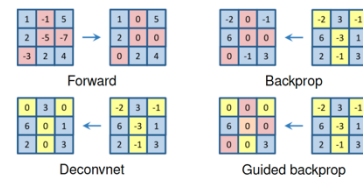
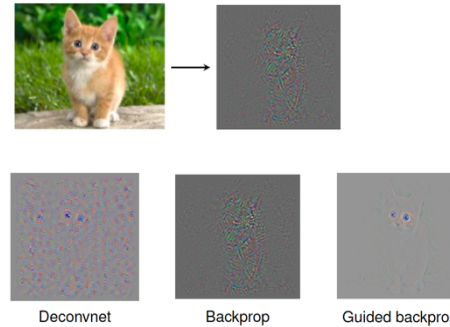
Simple Parameter Visualization

- Direct visualization of learned kernels: plot learned filter weights directly
 - Easy to implement and interpret for the first layer. If the first layer is very noisy, something is wrong with the setup
 - Higher layers are hard to interpret
- Visualization of the activations: tell us what the network is computing from a specific input
 - Strong response: feature is present, weak response: feature is absent
 - For higher-level neurons the activation gets very coarse
 - Drawback: no precise information about why that specific neuron was activated / what caused the response, coarse representation
- Occlusion: move a masking patch around the input image (which removes information from the image at this area) and then visualize the confidence for a specific decision. If occlusion causes a significant drop in prediction confidence, the occluded area is important for the classification.
 - Can identify confounds, wrong focus
 - Occlusion can be investigated through heat maps
- Maximally activating images: look at patches shown to a network and order them by confidence in a specific neuron
 - You try to find the input that activates a specific neuron the most

- Benefits: easy to implement, “false friends” are comparatively easy to find
- Drawbacks: Neurons don’t necessarily have semantic meaning by themselves and are rather basic vectors of representations
- T-SNE (stochastic neighborhood embedding): understand which images the network regards as similar
 - Compute the activations of the last layer and group inputs with similar activations
 - You perform a dimensionality reduction of the last layer activations (are relevant for the classification and encode the feature representation of the network), which produces a 2D map
 - Images that are perceived similar by the neural network are located in a direct neighborhood (clustered) in the dimensionality reduced representation

Gradient-Based Visualization

- Which input pixel is most significant to a neuron and what change would cause a large variation in the output
- Compute the partial derivative with respect to the input (backpropagation through the entire network)
- Visualize this gradient as a type of image
- Backpropagation: we need a loss (use a pseudo loss, which is the activation of an arbitrary neuron of any layer, typically from the output layer)
- Alternative: “reverse network” Deconvnet
 - Input: trained network and some image
 - Then choose one activation and set all the others to zero
 - Build a reverse network (in reverse sequence) with unpooling steps
 - No training involved
 - Forward pass of reverse network is same as backward pass apart from ReLU
- Guided backpropagation
 - Want to find positively correlated features (positive gradient = feature the neuron is interested in)
 - Set all negative gradients in the backpropagation to zero
 - Is a union of backpropagation and Deconvnet, it remembers the forward pass and sets all of those elements to zero and it sets all elements of the sensitivities to zero -> only keeps very few values
 - Backpropagation without guidance only sets the negative values from the forward pass to zero, Deconvnet only sets all the negative entries from the sensitivity to zero (not forward pass)
 - Deconvnet: noisy gradient, Backpropagation: focuses on the object of interest, guided backpropagation: very sparse representation and the most important features are clearly visible
- Saliency maps: investigate the impact of pixels on a class score instead of neurons
 - Pseudo loss is an unnormalized class score
 - Build the gradient with respect to the image pixels and use absolute values
 - Localizes things in the image even though the network wasn’t trained for localization



Parameter Visualization via Optimization

- Optimization on different levels (neuron, activation map, layer, logits, class probability)
- Changes the input such that different neurons are maximally activated (Inceptionism)
- Start with the image or noise as input, instead of adjusting network parameters, tweak image towards high activations of a complete layer
- For different layers it highlights different things in the image
- For random input you need an additional regularization (to postulate that the output image is smooth)
- Use it to reveal hidden weaknesses in the classification problem
- Inversion tries to invert from that activation what was the actual input (input is unknown)
- Solve a minimization problem such that we find with some input X the best match to that specific activation
- Add an additional regularizer for a more stable output (regularizer uses properties of a natural image)
- High frequency noise degrades the reconstruction, use filters to penalize it (regularizer)
 - TV (total variation) enforces image to have a low number of gradients (edges) / encourages sparse gradients
 - Low-pass filters can be applied in every iteration to x
 - Edge preserving filters
 - Is simple and effective, but also suppresses legitimate high frequency features
- Transform robustness: input should be invariant to spatial transformations, like data augmentation (regularizer)
 - Randomly rotate, scale or jitter x
 - Simple, effective, but orientation is suppressed even if it was informative
- Learned priors: enforces a prior on x to look like a natural image (regularizer)
 - Needs a trained generative model
 - Very nice images, but ambiguous whether to attribute features in result to the prior or the network
- Anonymizing data by cutting off the first to layers doesn’t work, because it can be reconstructed by inversion

Attention mechanisms

- Humans process data actively by shifting their focus
- The way how you shift your attention may change the interpretation
- Only focus on the relevant information
- We only want to look at the pixels that are relevant for our decision in order to make that decision
- Networks learn attention implicitly

- Can explicit attention mechanisms boost the performance?
- By plotting the gradients you can see the gradient with respect to that particular input of the respective output
Figure out which parts of the input sequence are related to which parts of the output sequence
- Sequence to sequence models
 - o Forward pass in encoder network receives some input sequence and computes the hidden states h (needs to process the entire sequence to compute the hidden state)
 - o Hidden state h (representation of the state/actual meaning) is used as a context vector for the decoder network and is then decoded into new sequence
 - o The decoder generates a new sequence of own hidden states s and an output sequence
 - o Different parts of input relate to different parts of output
 - o Encoding can be difficult, because entire meaning needs to be encoded into one context vector
- Idea: provide access to all contexts using a dynamic context vector as combination of all previous hidden states (former context vector did not allow access to earlier inputs, because it is obtained at the end of the sequence)
- Use a bidirectional encoder to gain access to all hidden states, which can be concatenated
- Then do a linear combination with some weights α = weighted average of the different hidden states
- Additional dynamic context for decoder: multiply hidden states with α and add them up
- Generate weights α using a dynamic procedure: applying a softmax operation to the previously computed alignment scores
- If current state is relevant for the observation in the decoding: high score
- Alphas encode the relevance of the current state with respect to the currently produced output
- Compute alignment score with a softmax function / cosine / (scaled) dot product / ...
- Or train an alignment score function
- Alignment score is computed by a comparison function that tells us which state is compatible with which other state. It takes the encoded hidden states and the previous decoder output to compute a score that indicates how well the elements of the input align with the current output at the position
- A unique context vector is provided for the decoder at each time step. It is computed by a weighted sum of all encoder states.
- Show-attend-tell: automatic generation of image captions, different elements in the image trigger different words
 - o Attention mechanism is used to improve the quality of the caption
- Soft attention:
 - o Model is fully differentiable
 - o Not effective/efficient for large inputs
- Hard attention: fix on glimpses / patches of the input
 - o Lower computation time
 - o Not differentiable (needs reinforcement learning)
- Soft attention produces fuzzy attention maps, hard attention is more localized
- Self attention
 - o Computes attention of sequence to itself
 - o Enrich representation of the tokens with context information
 - o Useful for machine translation / question answering / reasoning
 - o Relates parts of the input with other parts of the input
- AIAYN: attention is all you need
- Machine translation is only based on attention
- Iteratively improve representation by self-attention -> Transformer architecture
- Core blocks: Self-attention step and Local fully connected layer to merge the different attention heads (per token)
- Encoder learns an embedding algorithm: For each token a query, key and value is computed using trainable weights. Alignment between query and key is a scaled dot product that determines the influence of elements on the value: $\text{Attention}(q,k,v) = \text{softmax}(qk) * v$
- Attention blocks are stacked to have an additional positional encoding
- Allows for integration of knowledge independent of distance
- Positional encoding still allows to learn convolution-like steps
- Extremely versatile
- Extensions allow pretraining on unlabeled text
- Attention is based on the idea that you want to align or find out the relevance of input elements with respect to specific output elements
- Attention scores allow interpretation
- Allows to reformulate non-sequential tasks as sequential ones.
- Attention alone is very powerful because it's a transformer mechanism
- Very important for natural language processing tasks
- Attention is also used in combination with convolutions, but might also be seen as replacement for convolutions
- The idea behind attention mechanisms is to permit the decoder to use the most relevant parts of the input sequence in a flexible manner, by a weighted combination of all the encoded input vectors, with the most relevant vectors being attributed the highest weights.

Questions:

- Why is visualization important? To explain network to other people and yourself, to understand the network and identify issues.
- Why are confounds a problem? Network learns the wrong features -> classification doesn't work.
- Why could adversarial examples pose a security problem? Are generated to create a specific mistake, can for example confuse face recognition software.
- How does occlusion work? By hiding part of the data and then testing if the classification still works
- What is the difference between deconvolution, backpropagation and guided backpropagation regarding feature visualization? Backpropagation without guidance only sets the negative values from the forward pass to zero, Deconvnet only sets all the negative entries from the sensitivity to zero and guided back-propagation sets both to zero.
- How is Google DeepDream related to visualization? Changes picture to find things (towards the maximal activation) uses the technique of Inceptionism
- What is a Saliency map? Investigates the impact of pixels instead of neuron on a class score, is able to localize things in the image even though it wasn't trained for it
- What is the idea behind attention? Attention is based on the idea that you want to align or find out the relevance of input elements with respect to specific output elements.
- What is the difference between "normal" and self-attention? Normal: Relation input – output vs self: input – input
- How does the transformer architecture avoid recurrence and convolutions? By iteratively improving the representation with self attention. Therefore it only needs self attention and one fully connected layer. The self attention step replaces those layers.

Deep Reinforcement Learning

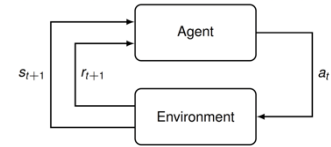
Sequential decision making

- Action: Formalize choosing an action a at time t from a set A (of possible actions)
- Reward: Action a_t has a different unknown pdf $p(r|a)$ generating reward r_t (PDF described relation between a & r)
- Policy: Formalize choosing an action a as pdf $\pi(a)$ which we call a policy. Rule that determines which a_t we choose. Pdf that describes the likelihood of choosing some action.
- Goal: find action a producing the maximum expected reward over time t : $\max_a \mathbb{E}[p(r|a)]$
- Difference to supervised learning: No feedback on what action to choose. Instead determine by our training algorithm which action to choose.
- $\mathbb{E}[p(r|a)]$ is not known in advance
- We can form a one-hot encoded vector r which reflects which action a caused the reward
- Action value function $Q_t(a) = \frac{1}{t} \sum_{i=1}^t r_i$ changes with every new information. It describes the value of taking an action at some state s when following policy π .
- $Q_{t+1}(a) = Q_t(a) + \frac{1}{t} (r_t - Q_t(a))$
- Exploitation: Reward is maximized by a policy choosing $\max_a Q_t(a)$
- Greedy action selection: if an action is good, we will exploit it further. Always choose the action with the highest reward.
- Greedy action selection always leads to the same results -> not suitable for training
- Sometimes we need to explore by selecting other moves (not with maximal reward) which could potentially be better in the long run
- Exploitation: always follow the path with the highest reward (greedy action selection). Always leads to the same results -> no learning possible
- Exploration: Sometimes take paths that don't have maximal reward, but might lead to higher overall reward.
- Different policies:
 - o Uniform random distribution: $\pi(a) = \frac{1}{|A|}$
 - choose action randomly with equal probability
 - good for exploration, but yields a lot of mistakes
 - o Epsilon Greedy: $\pi(a) = \begin{cases} 1 - \epsilon & \text{if } a = \max_a Q_t(a) \\ \frac{\epsilon}{n-1} & \text{else} \end{cases}$
 - 90% action that produces maximal expected reward, 10% for exploration
 - o Softmax: $\pi(a) = \frac{e^{Q_t(a)/\tau_t}}{\sum_{a \in A} e^{Q_t(a)/\tau_t}}$
 - τ_t is called temperature and used to decrease exploration over time

Reinforcement learning

- Network plays a game over and over again. Everytime it wins it remembers the gradients to use them in the future. Actions that lead to a positive reward are used more often, whereas action with a negative reward are used less frequently.
- Rewards can additionally depend on the state s_t : $p(r_t|s_t, a_t)$
- Whatever action we take will have an impact on the state $p(s_{t+1}|s_t, a_t)$

- Markov decision process
 - o Action: an action a_t at time t from set A
 - o State: a state s_t from a set S , state transition pdf $p(s_{t+1} | s_t, a_t)$
 - o Reward: Transition produces rewards r_{t+1} according to $p(r_{t+1} | s_t, a_t)$
 - o Policy: Agents choose actions a_t by a policy
 - o Actions have influence on environment $\pi(a | s)$
 - o If all those sets are finite we call this a finite MDP
- Good policy: goal is to maximize future return



Policy iteration

- State-value function $v_\pi(s)$ tells us how valuable a state is. It is the expected return/reward when starting in this state s and acting according to our policy
- Action-value function $Q_\pi(s, a)$ describes value of training an action a in some state s when following a policy. It is the expected return/reward given the state s and action a under a policy. It accounts for the transition probabilities
- Optimal $V^*(s)$ is maximum overall state-value function $V(s)$ with best policy. Best policy always produces $V^*(s)$
- V^* is strictly positive and deterministic
- Policy π with $V_\pi = V^*$ is an optimal policy π^*
- There can be more than one optimal policy
- Bellman equations are used to compute optimal value- and state-functions
- Policy improvement theorem: iteratively updating V with greedy action selection, will always generate a better policy, which results in an increasing V
- Limitations: state-transition pdf and reward pdf need to be known
- On-policy algorithms: require using the updated policies during learning to obtain better approximations to $V^*(s)$
- Learning techniques
- Monte Carlo technique
 - o Only for episodic tasks
 - o Off-policy: learns $V^*(s)$ by following any random (s, a)
 - o Does not need information about dynamics of the environment
 - o Generate an episode by using some policy
 - o Loop backwards over the episode accumulating the expected future reward $g_t = g_{t+1} + r_{t+1}$
 - o If a state was not yet visited append g_t to a list $returns(s_t)$
 - o Update $V_{s_t} = \frac{1}{N} \sum_{n=1}^N returns_n(s_t)$
- Temporal difference learning
 - o On-policy, does not need information about dynamics of the environment
 - o Updates V with the weighted influence of new observations
 - o Converges to the optimal solution
- Q learning
 - o Off-policy, temporal difference type of method, does not need information about dynamics of the environment
 - o Loop and follow $\pi(s_t, a_t)$ derived from $Q_t(s, a)$: ϵ -greedy
 - o Use a from $\pi(s_t, a_t)$, observe r_t, s_{t+1}
 - o Update $Q_{t+1}(s, a) = Q_t(s_t, a_t) + \alpha [r_t + \gamma \max_a Q_t(s_{t+1}, a_t) - Q_t(s_t, a_t)]$
- REINFORCE: policy gradient
 - o Parametrize $\pi(s_t, a_t, w)$ with weights w and use some loss-function L

Deep Q learning

- Deep Q-network: deep network that applies Q learning
- Each output estimates optimal action value for its action given the input
- Instead of label & cost, update to maximize reward
- Using ϵ -greedy policy with a low ϵ
- Semi-gradient form of Q-learning to update network weights w
- Uses mini-batches to accumulate weights updates
- Target changes simultaneously with the weights we want to learn
- Training can oscillate or diverge
- Idea: use a second target network:
 - o After each C steps, copy weights of action-value network to a duplicate network and keep them fixed
 - o Use output of target network to stabilize
- Experience replay: store actions in replay memory and then use them later again. Removes dependence on current weights and increases stability

Mastering Go

- Combination of multiple methods: deep neural networks, Monte Carlo tree search, supervised learning and reinforcement learning
- Monte Carlo tree search: look ahead a couple of steps and evaluate those
 - o Selection: starting at root, traverse with tree policy to a leaf node

- Expansion: (optional) add one or more child nodes to the current leaf
 - Simulation: from the current or the child node, simulate episode with action according to rollout policy
 - Backup: propagate the received reward back through the tree
- Utilization of three different networks
 - Policy network: suggests the next move in leaf nodes for extension. Start with supervised learning (train network on human expert plays), then use reinforcement learning (play against older version of itself)
 - Value network: given the current board position, get chances of winning. Estimates how likely the current state leads to a win. Train with reinforcement learning and Monte Carlo policy.
 - Rollout policy network: guide rollout action selection. Simpler than policy network, but much faster.

Questions

- What is a policy? Rule that determines how the agent chooses actions. Input: state and action, output probability of choosing this state.
- What are value functions? Describes the value of an action or state if a certain policy is used.
- Explain the exploitation vs exploration dilemma. Exploitation: always choose path with highest reward. Exploration: sometime choose a different path that leads to overall higher reward.
- Describe typical solutions to the dilemma. Epsilon-greedy function.
- What is the difference of a multi armed bandit problem to the full reinforcement learning problem? Bandit: reward only depends on action, no state involved.
- Describe a Markov decision process.
- Is an optimal policy necessarily unique? No several policies can lead to optimal state value and are therefore all optimal.
- What do the Bellman equations represent? Optimization principle that says that a complex problem can be split into smaller problems.
- Describe policy iteration. Iteratively improving the policy, start with random policy. Update policy if result is better.
- Why does policy iteration work? Because of the policy improvement theorem.
- How can you beat your friends in every Atari game?
- How can one master the game of Go?

Unsupervised Deep Learning

- Motivation
 - Medical data: small dataset (data highly sensitive), one complex object, many modalities
 - Problem: data annotation is expensive to obtain
- Weakly supervised learning: label for related task
- Semi-supervised learning: partial (little) labeled data
- Unsupervised learning: no labeled data
- Generative model: is a statistic model of the joint probability distribution $P(X,Y)$ on given observable variable X and target variable Y
- Discriminative model: is a model of the conditional probability $P(Y|X=x)$ of the target Y , given observation x_i

Autoencoders

- Non-linear dimensionality reduction
- Special case of feed-forward neural network
- AE tries to learn an approximation of the identity
- The reason behind using an AE is that we want to understand and represent only the deep correlations and relationships among data
- Why use autoencoder and not PCA -> PCA is linear, AE is nonlinear
- The input is squeezed down to a lower encoded representation using an encoder network, then a decoder network decodes the encoding to recreate back the input
- Encoder: The encoder is the part of the network which takes in the input and produces a lower dimensional encoding
- Bottleneck: It is the lower dimensional hidden layer where the encoding is produced. The bottleneck layer has a lower number of nodes (undercomplete AE) and the number of nodes in the bottleneck layer also gives the dimension of the encoding of the input
- Decoder: The decoder takes in the encoding and recreates back the input
- Undercomplete AE: smaller hidden layer
- Sparse AE: larger hidden (bottleneck) layer. Enforces sparsity in the activations
 - Regularize the activations not weights, when activation is 0, the node is not contributing information -> smaller network
 - KL (Kullback-Leibler) divergence: a way to measure the difference and similarity between two mathematical probability distributions
- Convolutional AE: replace fully connected layer with convolutional layer (optionally add pooling layers)
- Denoising AE: corrupt the input, then denoising and regularization
 - Only look at the object image and not the noise, which is eliminated
- Stacked AE: 2 or more AEs stacked on one another or multiple layers in one AE

- Variational: instead of computing a deterministic feature vector describing the attributes of the input in latent space, it uses a variational approach to learn the latent representation.
 - o Describes observation in latent space/latent attribute in a probabilistic manner.
 - o Allows to model uncertainty in the input data
 - o Decoding: sample from latent space as input for decoder model
 - o Problem: no backpropagation possible
 - o Probabilistic models -> allow to generate data
 - o Intractable density -> optimize variational lower bound instead
 - o Trained via back propagation by using reparametrization
 - o Pros: principled approach to generative models, latent space representation can be useful for other tasks
 - o Cons: Only maximizes lower bound of likelihood, samples in standard models often of lower quality compared to GANs

Generative Adversarial Networks

- An unsupervised learning task in ML that involves automatically discovering and learning the regularities or patterns in input data in such a way that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset
- Generator (trained to generate new examples) + Discriminator (tries to classify examples as real or fake)
- Training GANs:
 - o Train D: minimize $L^D(\theta^D, \theta^G) = -\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \log D(\underbrace{\mathbf{x}}_{\text{real}}) - \mathbb{E}_{\mathbf{z} \sim p_z} \log(1 - D(\underbrace{G(\mathbf{z})}_{\text{fake}}))$
 - o Train G: minimize $L^G = -L^D$
 - minimizes log probability of the discriminator of being correct
 - o Minmax game: $\hat{\theta}^G = \arg \min_{\theta^G} \max_{\theta^D} V(\theta^D, \theta^G)$
- Evaluation of GAN with inception score
 - o Generated image should be recognizable (score distribution should be dominated by one class, image-wise class distribution should have low entropy)
 - o Generated images should be diverse (class distribution uniform, entropy should be high)
- Conditional GAN
 - o Idea: provide additional vector y to networks to encode conditioning
 - o Generator/Discriminator learn to operate in modes
- Cycle Consistent GANs: Couple GAN with trainable inverse mapping F
- One-sided Label smoothing: replace targets of the real samples with a smoothed version, do not do the same for fake samples
 - o Prevents D from giving very large gradient signal to G
 - o Prevents extrapolating to encourage extreme samples
- Virtual Batch normalization: create reference batch R of random samples chosen and fixed once at the start of training
 - o For each $\mathbf{x}_i$ of the current mini-batch create a new virtual mini batch $V_i = R \cup \mathbf{x}_i$
 - o BN causes strong intra-batch correlation, use VBN instead
- Historical averaging: add penalty term that punishes weights which are rather far away from their historical average
- GANs for semi-supervised learning
 - o Idea: Use GANs by turning the problem from a K class problem into a problem with K + 1 classes
 - o True classes: target classes for supervised learning
 - o Additional class: fake inputs generated by the generator
 - o Probability of image being real: sum of real class probabilities
 - o Discriminator is used as a classifier within the GAN game

Summary

- GANs are generative models that use supervised learning to approximate an intractable cost function
- GANs can simulate many cost functions
- Hard to find equilibrium between Discriminator and Generator
- Can't generate discrete data
- Can also be used for (semi-) supervised classification, transfer learning, multi-modal outputs

Questions

- What is the defining characteristic of an autoencoder? AEs try to learn an approximation of the identity.
- How do denoising autoencoders work? Corrupt the input with noise -> target = the input without noise. Additional regularization similar to dropout.
- What does an optimal discriminator for GANs learn? Key assumption: both densities are non-zero everywhere, $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{model}}(x)}$. The optimal is unachievable: the key for GANs is to have an approximation mechanism.
- What are the advantages of GANs in comparison to other generative models? Ability to generate samples in parallel, very few restrictions, no Markov chain needed, no variational bound is needed, GANs are known to be asymptotically consistent since the model families are universal function approximators.

- What is “mode collapse”? G rotates through the modes of the data distribution and never converges to a fixed distribution. Reason: D in inner convergence to correct distribution, but G places all mass on most likely point. Solution: Minibatch discrimination or unrolled GANs
- Explain feature matching / perceptual loss. G is trained to match the expected value of features $f(x)$ of intermediate layer of D. This prevents “Overtraining” of G on current D.